

ResearchGraph 数据库课程大作业报告

项目名称: ResearchGraph —— 融合 Zotero 式文献管理、Obsidian 式双链笔记与 AI 知识图谱分析的本地科研协作系统

技术栈:

- Vue 3 + Vite + Element Plus + Cytoscape.js (前端)
- FastAPI + SQLAlchemy + SQLite (后端)
- scikit-learn TF-IDF + sentence-transformers (AI 引擎) + OpenAI SDK

目录

- 绪论
- 需求分析
- 数据库设计
- 系统实现
- SQL 查询与数据库特性
- 系统测试与展示
- 小组分工
- 总结与展望

1. 绪论

1.1 选题背景

科研小组在文献调研中普遍面临以下痛点:

- 论文数量多、来源杂, 难以统一管理;
- PDF、BibTeX、阅读笔记分散在不同位置, 缺乏关联;
- 组内成员之间信息不互通, 不清楚彼此读过哪些论文;
- 论文之间的引用、方法、主题关系不直观, 难以形成知识网络;
- 撰写 Related Work 时, 难以快速定位某一方向下的代表性论文;
- 传统文献工具 (如 Zotero) 偏重"条目管理", 知识网络与协作能力较弱。

本系统借鉴 **Zotero** 的"结构化元数据数据库 + 本地文件存储"思想, 同时融入 **Obsidian** 的双向链接理念, 将论文、笔记、概念、标签、作者、项目组织成一张知识图谱, 并以一个轻量、可解释的 AI 模块自动完成标签推荐、相似论文发现与关系补边, 最终用 Cytoscape.js 进行可视化。

1.2 系统目标

实现一个面向数据库课程展示的最小可行科研协作系统，支持：

- Zotero 式文献元数据 / 作者 / 会议 / 标签 / 附件 / BibTeX 管理；
- Obsidian 式 Markdown 双链笔记与概念卡片；
- AI 自动标签、相似论文推荐、知识图谱关系补边、Related Work 主题聚类；
- 科研项目协作：成员、角色、阅读任务、评论、活动日志；
- 知识图谱数据生成与前端可视化；
- 满足数据库课程要求的表设计、约束、视图、触发器、索引、事务、复杂查询等特性。

1.3 创新点

创新点	说明
三种工具理念融合	将文献管理（Zotero）、双链笔记（Obsidian）、AI 图谱分析整合到同一数据模型中，统一以"知识图谱"视角组织。
AI 分析 (local / cloud 双模式)	AI 模块通过 AIProvider 抽象出 embed / suggest_tags / suggest_relations / name_cluster 接口，本地用 sentence-transformers，离线时自动降级为纯 TF-IDF，亦可一键切换 OpenAI 兼容云端 API，算法完全可解释。
AI 建议与正式数据分离 + 人工确认闭环	AI 产物先写入 ai_tag_suggestions / ai_relation_suggestions 建议表，用户确认后才在事务中落入 tags/paper_tags/paper_relations 正式表，保证数据可控。
统一图谱端点	单个 /api/graph 接口聚合 7 类节点、12+ 类边，支持按项目、论文、概念、节点类型、边类型多维筛选。
向量缓存机制	paper_embeddings 表以 BLOB 缓存论文向量，并用 text_hash 判断缓存是否过期，相似度计算与聚类可复用，避免重复 embedding。

2. 需求分析

2.1 用户角色分析

系统设三类角色，权限逐级递减；登录逻辑简化为本地账号密码（SHA-256 散列）：

角色	权限
Admin	管理项目、成员、论文、任务、关系图谱
Member	添加论文、写笔记、确认 AI 推荐、完成阅读任务
Viewer	查看论文、笔记、图谱与任务进度

角色记录在 project_members.role 中，并由 CHECK(role IN ('admin','member','viewer')) 约束。

2.2 文献管理需求 (模块 A)

论文及其元数据 (标题、摘要、年份、会议、DOI、arXiv、阅读状态)、作者、会议/期刊、标签、Collection、PDF 附件、BibTeX 的增删改查; 论文列表支持按标题搜索、按标签/年份/会议/阅读状态筛选与分页; 论文详情聚合作者、标签、会议、笔记、附件、BibTeX。

2.3 双链笔记需求 (模块 B)

为论文创建 Markdown 阅读笔记, 支持 `[[概念名]]` 与 `[[笔记标题|别名]]` 形式的双向链接; 后端用正则提取双链, 自动创建概念卡片并建立 note-concept、note-note 关系; 提供反向链接 (哪些笔记链向当前笔记、哪些笔记与当前笔记共享概念)。

2.4 AI 分析需求 (模块 C)

四项 AI Analysis 功能:

1. **自动标签推荐**: 基于概念词典扫描论文标题/摘要, 给出标签、置信度、命中原因;
2. **相似论文推荐**: 拼接 title + abstract + notes, TF-IDF / 句向量编码后计算余弦相似度, 返回 Top-k;
3. **关系补边**: 依据相似度、年份先后、方法关键词重叠与"改进"措辞, 启发式推断 `same_topic` / `method_related` / `possible_baseline` / `compares_with` / `improves` 等关系类型;
4. **Related Work 聚类**: 对全库论文做 KMeans 聚类, 自动命名主题并给出每篇论文的隶属度。

2.5 科研协作需求 (模块 D)

用户登录; 创建科研项目、添加成员并设角色; 分配论文阅读任务、更新任务状态 (todo→reading→done→reported); 对论文/笔记/任务评论; 记录活动日志; 生成并可视化知识图谱, 支持节点/边类型筛选、搜索、点击查看详情、缩放拖拽。

3. 数据库设计

3.1 概念结构设计 (E-R 概览)

核心实体与联系如下 (→ 表示一对多, ↔ 表示多对多):

```

User ↔ Project (project_members, 带 role)      Project → ReadingTask →
TaskAssignment ↔ User
Paper ↔ Author (paper_authors, 带 author_order)  Paper ↔ Tag (paper_tags)
Paper → Venue                                  Paper ↔ Collection
(collection_papers)
Paper → Attachment / BibtexEntry / Note          Note ↔ Concept
(note_concepts)
Note ↔ Note (note_links, 自关联)                Collection → Collection
(parent_id, 自关联)
Paper ↔ Paper (paper_relations / paper_similarity, 自关联)
ResearchCluster ↔ Paper (cluster_papers)        Paper → PaperEmbedding(向量
缓存)

```

3.2 逻辑结构设计

数据库实际包含 **28 张表**（含多对多中间表），按模块归类：

模块	数据表
A 文献管理	papers authors paper_authors venues tags paper_tags collections collection_papers attachments bibtex_entries
B 双链笔记	notes concepts note_concepts note_links
C AI 分析	ai_tag_suggestions paper_similarity ai_relation_suggestions paper_relations research_clusters cluster_papers paper_embeddings
D 协作图谱	users projects project_members reading_tasks task_assignments comments activity_logs

3.3 表结构设计（核心表举例）

论文表 **papers**（部分字段）：

```
CREATE TABLE papers (
    paper_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    title         TEXT NOT NULL,
    abstract      TEXT,
    year          INTEGER,
    venue_id      INTEGER,
    paper_type    TEXT DEFAULT 'article',
    doi           TEXT,
    arxiv_id      TEXT,
    url           TEXT,
    reading_status TEXT DEFAULT 'unread'
    CHECK(reading_status IN ('unread', 'reading', 'finished', 'archived')),
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (venue_id) REFERENCES venues(venue_id)
);
```

论文关系表 **paper_relations** (自关联, AI 补边落地表) :

```
CREATE TABLE paper_relations (
    relation_id   INTEGER PRIMARY KEY AUTOINCREMENT,
    source_paper_id INTEGER NOT NULL,
    target_paper_id INTEGER NOT NULL,
    relation_type TEXT NOT NULL
    CHECK(relation_type IN
('cites', 'same_topic', 'method_related', 'extends',
' improves', 'compares_with', 'contradicts', 'baseline_of', 'uses_method_of')),
    weight        REAL DEFAULT 1.0,
    generated_by  TEXT DEFAULT 'user',    -- user / ai
    confidence    REAL DEFAULT 1.0,
    is_confirmed  BOOLEAN DEFAULT 1,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (source_paper_id) REFERENCES papers(paper_id),
    FOREIGN KEY (target_paper_id) REFERENCES papers(paper_id)
);
```

3.4 主键、外键与约束

- **主键**: 单列自增主键 (如 `paper_id`) + 复合主键 (如 `paper_authors(paper_id, author_id)`、`task_assignments(task_id, user_id)`)。
- **外键**: 所有中间表与从属表均声明外键, 保证参照完整性; 从属表通过 SQLAlchemy `cascade="all, delete-orphan"` 实现级联删除 (如删除论文时连带删除附件、BibTeX)。
- **唯一约束**: `users.username`、`tags.tag_name`、`concepts.concept_name`、`venues.venue_name`、`authors.author_name` 等保证去重。
- **CHECK 约束**: `reading_status`、`role`、`relation_type`、`venue_type`、`note_type`、`task_assignments.status`、`comments.target_type` 等枚举字段均有取值约束。

- 自关联结构 (≥2 个) : ① paper_relations / paper_similarity 论文→论文; ② note_links 笔记→笔记; ③ collections.parent_id 集合→父集合。

3.5 视图、索引、触发器设计

视图 (2 个)

SQL

```
-- 视图 1: 论文详情视图, 把论文 + 会议 + 作者 + 标签聚合为一行
CREATE VIEW paper_detail_view AS
SELECT p.paper_id, p.title, p.year, p.abstract, v.venue_name,
       GROUP_CONCAT(DISTINCT a.author_name) AS authors,
       GROUP_CONCAT(DISTINCT t.tag_name) AS tags
FROM papers p
LEFT JOIN venues v ON p.venue_id = v.venue_id
LEFT JOIN paper_authors pa ON p.paper_id = pa.paper_id
LEFT JOIN authors a ON pa.author_id = a.author_id
LEFT JOIN paper_tags pt ON p.paper_id = pt.paper_id
LEFT JOIN tags t ON pt.tag_id = t.tag_id
GROUP BY p.paper_id;

-- 视图 2: 项目任务进度视图, 按项目 + 成员统计完成情况
CREATE VIEW project_task_progress_view AS
SELECT p.project_id, p.project_name, u.username,
       COUNT(ta.task_id) AS total_tasks,
       SUM(CASE WHEN ta.status IN ('done', 'reported') THEN 1 ELSE 0 END) AS
finished_tasks
FROM projects p
JOIN reading_tasks rt ON p.project_id = rt.project_id
JOIN task_assignments ta ON rt.task_id = ta.task_id
JOIN users u ON ta.user_id = u.user_id
GROUP BY p.project_id, u.user_id;
```

索引 (≥5 个, 实际 7+ 业务索引 + 主键索引)

SQL

```
CREATE INDEX idx_papers_title ON papers(title);
CREATE INDEX idx_papers_year ON papers(year);
CREATE INDEX idx_paper_tags_tag_id ON paper_tags(tag_id);
CREATE INDEX idx_paper_authors_author_id ON paper_authors(author_id);
CREATE INDEX idx_notes_paper_id ON notes(paper_id);
CREATE INDEX idx_attachments_paper_id ON attachments(paper_id);
CREATE INDEX idx_bibtex_entries_paper_id ON bibtex_entries(paper_id);
```

触发器 (2 个)

```

-- 触发器 1: 论文被更新后自动刷新 updated_at
CREATE TRIGGER trg_update_paper_timestamp
AFTER UPDATE ON papers FOR EACH ROW
BEGIN
    UPDATE papers SET updated_at = CURRENT_TIMESTAMP WHERE paper_id =
    OLD.paper_id;
END;

-- 触发器 2: 任务状态被改为 done 时自动写入活动日志
CREATE TRIGGER trg_task_done_log
AFTER UPDATE ON task_assignments FOR EACH ROW
WHEN NEW.status = 'done' AND OLD.status <> 'done'
BEGIN
    INSERT INTO activity_logs(user_id, action_type, target_type, target_id,
description)
    VALUES (NEW.user_id, 'TASK_DONE', 'task', NEW.task_id, 'User completed a
reading task');
END;

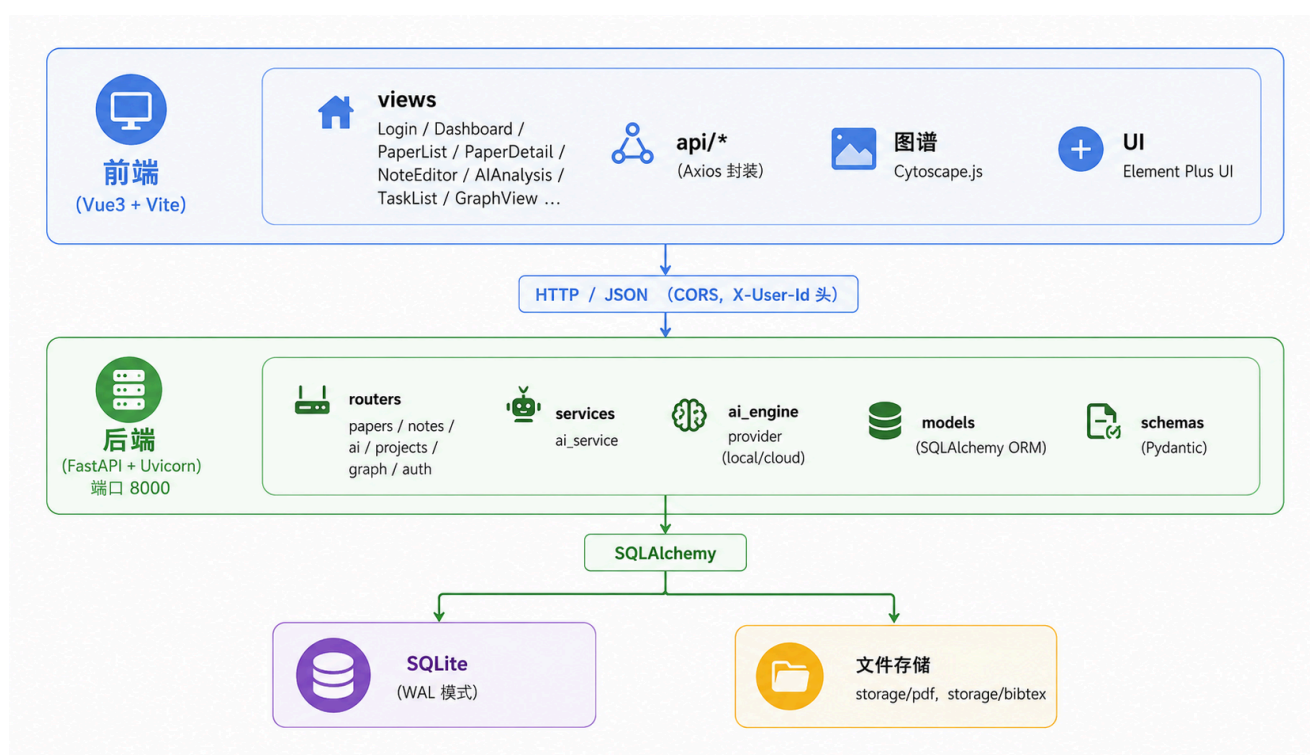
```

视图、索引、触发器均在 `database.py::init_sqlite_schema()` 中以 `CREATE ... IF NOT EXISTS` 等创建，应用启动时自动执行，无需手动建库。同时启用了 `PRAGMA journal_mode=WAL + busy_timeout=30000` 提升读写并发、缓解 "database is locked"。

4. 系统实现

4.1 系统架构

采用前后端分离的三层架构：



后端按 `models / schemas / routers / services / ai_engine` 分层；`main.py` 注册各路由前缀（`/api/papers`、`/api/ai`、`/api/notes`、`/api/projects`、`/api/graph` 等），并开启 CORS。FastAPI 自带 `/docs` Swagger UI 可直接联调测试。

4.2 文献管理模块 (A)

- **CRUD**：`routers/papers.py` 提供论文增删改查；列表接口支持 `keyword / tag_id / tag / year / venue_id / reading_status` 多条件过滤与 `skip/limit` 分页，并用 `joinedload` 预加载作者、标签、会议、附件等关联，避免 N+1 查询。
- **作者/标签/会议去重**：`_get_or_create_author/tag/venue` 以规范化名称（去空白、casefold）实现幂等“存在即取、不存在则建”。
- **BibTeX 导入（事务）**：用 `bibtexparser` 解析 `.bib`，逐条解析作者（`split(' and ')`）、关键词、会议、年份，写入 `papers / venues / authors / paper_authors / tags / bibtex_entries`，整体置于 `try/except` 中，任一步失败 `db.rollback()` 全部回滚，全部成功统一 `db.commit()`。
- **PDF 附件**：上传文件以 `{paper_id}_{uuid}.pdf` 存入 `storage/pdf`，数据库仅存相对路径；预览接口做了路径穿越防护（`is_relative_to(storage_root)`）。

4.3 双链笔记模块 (B)

核心是 `routers/notes.py::_sync_note_links()`：

1. 用正则 `\\[\\([\\^\\[\\]\\\\n]+\\)\\]` 提取笔记中的所有双链名，支持 `[[Target|Alias]]` 别名语法（取 | 前部分）并去重；
2. 先清空该笔记旧的 `note_concepts` 与 `note_links`（覆盖式同步）；
3. 对每个链接名：概念不存在则自动创建（`concepts`），并在 `note_concepts` 建立 `note→concept` 关系（`source='wikilink'`）；
4. 若存在同名笔记标题，则在 `note_links` 建立 `note→note` 双链；
5. 笔记创建 / 更新 / 调用 `parse-links` 时均触发同步。

反向链接接口 `GET /notes/{id}/backlinks` 返回两类：① 直接链向当前笔记的 inbound 笔记；② 与当前笔记共享概念的笔记。

4.4 AI 分析模块 (C)

引擎抽象：`ai_engine/provider.py` 定义统一接口，`LocalProvider`（离线）与 `CloudProvider`（OpenAI 兼容）实现之；`services/ai_service.py` 负责把引擎接到 SQLAlchemy 表上，并做字段映射（`score→confidence`、`src/dst→source/target_paper_id`、`mode→model_name`）。

- **标签推荐**：`LocalProvider.suggest_tags` 用概念词典 `LEXICON` 扫描命中次数，`score = min(1.0, 0.5 + 0.15*(count-1))`，给出命中关键词作为 `reason`，写入 `ai_tag_suggestions`；用户确认时在事务中写 `tags + paper_tags` 并置 `is_accepted=1`。
- **相似论文**：`_corpus()` 一次性载入全库论文文本（TF-IDF 语料相关，必须整库重算），编码为矩阵并缓存进 `paper_embeddings`；`_cosine_matrix()` 计算余弦相似度，取 Top-k

(过滤 ≤ 0.1) 覆盖式写入 `paper_similarity`。

- **关系补边**：以相似度阈值（句向量 0.45 / TF-IDF 0.18）筛候选，按"年份先后 + 改进措辞 + 方法关键词重叠"启发式判定关系类型，写入 `ai_relation_suggestions`，确认后在事务中写 `paper_relations`（`generated_by='ai'`）。
- **Related Work 聚类**：对全库向量做 KMeans（`random_state=42`），按簇中心余弦相似度计算每篇隶属度，自动命名主题，写入 `research_clusters` + `cluster_papers`。

幂等设计：重新生成前清除"未被接受"的旧建议、跳过"已接受"的建议，避免重复堆积。

4.5 协作与知识图谱模块 (D)

- **认证**：`/api/login` 校验 SHA-256 散列密码；后续请求通过 `X-User-Id` 头识别当前用户（`get_current_user` 依赖注入）。
- **项目/成员/任务/评论**：完整 CRUD；创建项目时创建者自动成为 admin 成员；所有写操作均调用 `_log_activity()` 记录活动日志。
- **任务状态流转**：`PUT /tasks/{id}/status` 更新当前用户分配状态，置为 `done/reported` 时记 `finished_at`，并由触发器 `trg_task_done_log` 自动补一条日志。
- **Dashboard 统计**：聚合论文数、笔记数、项目数、待办任务数、最近论文、最近活动、高频标签。
- **知识图谱**：`GET /api/graph` 统一生成 `nodes/edges`——节点类型涵盖 `paper / author / venue / tag / concept / note / project / user / cluster`；边类型涵盖 `authored_by / published_in / has_tag / has_note / mentions_concept / links_to / assigned_to / belongs_to / semantic_similar` 及各 `paper_relations` 关系类型；支持 `project_id / paper_id / concept_id / node_type / edge_type` 筛选，并清理孤儿边。前端 `GraphView.vue` 用 `Cytoscape.js` 渲染，不同节点类型不同颜色/形状，支持拖拽、缩放、点击查看详情、搜索高亮。

5. SQL 查询与数据库特性

以下展示满足验收要求（ ≥ 6 条复杂查询、自关联、聚合、事务、触发器、索引）的代表性 SQL。

5.1 多表连接查询

① 论文完整信息（论文 + 会议 + 作者 + 标签）

```
SELECT p.paper_id, p.title, p.year, v.venue_name,
       GROUP_CONCAT(DISTINCT a.author_name) AS authors,
       GROUP_CONCAT(DISTINCT t.tag_name) AS tags
FROM papers p
LEFT JOIN venues v      ON p.venue_id = v.venue_id
LEFT JOIN paper_authors pa ON p.paper_id = pa.paper_id
LEFT JOIN authors a      ON pa.author_id = a.author_id
LEFT JOIN paper_tags pt  ON p.paper_id = pt.paper_id
LEFT JOIN tags t         ON pt.tag_id = t.tag_id
GROUP BY p.paper_id;
```

SQL

② 按标签筛选论文 (多对多连接)

```
SELECT DISTINCT p.paper_id, p.title, p.year
FROM papers p
JOIN paper_tags pt ON p.paper_id = pt.paper_id
JOIN tags t        ON pt.tag_id = t.tag_id
WHERE t.tag_name = 'flow-matching';
```

SQL

5.2 聚合统计查询

③ 高频标签 Top-10 (Dashboard 用)

```
SELECT t.tag_name, COUNT(pt.paper_id) AS cnt
FROM tags t
JOIN paper_tags pt ON t.tag_id = pt.tag_id
GROUP BY t.tag_id
ORDER BY cnt DESC
LIMIT 10;
```

SQL

④ 各项目成员任务完成进度 (含 CASE WHEN 条件聚合)

```
SELECT p.project_name, u.username,
       COUNT(ta.task_id) AS total_tasks,
       SUM(CASE WHEN ta.status IN ('done', 'reported') THEN 1 ELSE 0 END) AS
finished
FROM projects p
JOIN reading_tasks rt ON p.project_id = rt.project_id
JOIN task_assignments ta ON rt.task_id = ta.task_id
JOIN users u           ON ta.user_id = u.user_id
GROUP BY p.project_id, u.user_id;
```

SQL

5.3 自关联查询

⑤ 某论文的相似论文 Top-k (papers 表自连接)

```
SELECT ps.target_paper_id, tp.title, ps.similarity_score, ps.method
FROM paper_similarity ps
JOIN papers tp ON tp.paper_id = ps.target_paper_id
WHERE ps.source_paper_id = :pid
ORDER BY ps.similarity_score DESC
LIMIT 5;
```

SQL

⑥ 某概念被哪些笔记引用 (反向链接 / 双链网络)

SQL

```
SELECT n.note_id, n.title, p.title AS paper_title
FROM note_concepts nc
JOIN notes n ON n.note_id = nc.note_id
LEFT JOIN papers p ON p.paper_id = n.paper_id
WHERE nc.concept_id = :cid;
```

⑦ 笔记间双链 (note_links 自关联)

SQL

```
SELECT s.title AS source_note, t.title AS target_note, nl.link_type
FROM note_links nl
JOIN notes s ON s.note_id = nl.source_note_id
JOIN notes t ON t.note_id = nl.target_note_id;
```

5.4 事务实现

BibTeX 导入是典型的多表写入事务场景 (routers/papers.py) :

PYTHON

```
@import_router.post("/bibtex")
def import_bibtex_text(data, db):
    try:
        imported = _import_bibtex_entries(db, data.raw_bibtex) #
papers/venues/authors/paper_authors/tags/bibtex_entries
        db.commit() # 全部成功 → 统一提交
    except Exception as exc:
        db.rollback() # 任一步失败 → 全部回滚
        raise HTTPException(status_code=400, detail=f"Invalid BibTeX:
{exc}")
```

AI 标签 / 关系确认同样以事务保证"建议表标记 is_accepted"与"正式表插入"原子完成。

5.5 触发器实现

见 3.5 节两个触发器。验证: 调用 PUT /api/tasks/{id}/status 把任务置为 done 后, activity_logs 会自动多出一条 TASK_DONE 日志, 无需应用层额外插入。

5.6 索引优化

对高频过滤 / 连接列建索引: papers.title (标题搜索)、papers.year (年份筛选)、paper_tags.tag_id 与 paper_authors.author_id (多对多连接)、notes.paper_id (论文笔记) 等。可用 EXPLAIN QUERY PLAN 对比建索引前后是否由全表扫描 (SCAN) 转为索引查找 (SEARCH USING INDEX) 来佐证优化效果。

6. 系统测试与展示

6.1 测试数据

通过 `seed_papers.py` / `generate_ai_data.py` 注入了演示数据，当前数据库规模：

表	行数	表	行数
papers	20	paper_tags	58
authors	20	tags	16
paper_authors	36	concepts	14
venues	12	note_concepts	25
notes	10	paper_similarity	50
paper_relations	30	ai_relation_suggestions	35
ai_tag_suggestions	24	research_clusters / cluster_papers	3 / 20
paper_embeddings	20	users	1

数据库对象统计：**28 张表、2 个视图、2 个触发器、7 个业务索引（+ 主键索引）**，满足并超出" ≥ 15 表 / ≥ 2 视图 / ≥ 2 触发器 / ≥ 5 索引 / ≥ 3 个多对多 / ≥ 2 个自关联 / ≥ 6 条复杂查询 / ≥ 1 个事务"的全部验收指标。

6.2 使用说明

1. 安装依赖

SHELL

```
pip install -r requirements.txt

cd frontend
npm install

python seed_papers.py # (可选) 注入样例数据
```

2. 启动后端

SHELL

```
cd backend
copy .env.example .env # (可选) 配置 AI 模式: 复制 .env, 默认 local 离线即可, 无需改动
uvicorn main:app --reload
```

3. 启动前端

SHELL

```
cd frontend
npm run dev
```

启动后通过浏览器访问终端提示的地址（通常是 `http://localhost:5173`）

7. 小组分工

成员	模块	主要产出
汤沂昕	Zotero 式文献管理	<code>papers/authors/tags/venues/collections/attachments/bibtex_entries</code> 表、 <code>routers/papers.py</code> 、BibTeX 事务导入、PaperList / PaperDetail / BibtexImport 页面
邓熙涵	Obsidian 式笔记双链	<code>notes/concepts/note_concepts/note_links</code> 表、 <code>routers/notes.py</code> 双链解析、NoteEditor / NoteList / ConceptList / ConceptDetail 页面
李亚霖	AI 科研分析	<code>ai_tag_suggestions/paper_similarity/ai_relation_suggestions/research_clusters/cluster_papers/paper_embeddings</code> 表、 <code>ai_engine/*</code> 引擎、 <code>services/ai_service.py</code> 、AIAnalysis 页面
苑康鑫	协作与知识图谱	<code>users/projects/project_members/reading_tasks/task_assignments/comments/activity_logs/paper_relations</code> 表、 <code>routers/auth.py</code> / <code>projects.py</code> / <code>graph.py</code> 、视图与触发器、Login / Dashboard / ProjectDashboard / TaskList / GraphView 页面与系统集成

8. 总结与展望

8.1 工作总结

本项目完整实现了一个融合文献管理、双链笔记、AI 分析与协作图谱的本地科研系统。在数据库层面，设计了 28 张规范化数据表，综合运用主外键、唯一/CHECK 约束、多对多与自关联结构、视图、索引、触发器、事务等特性，覆盖并超出课程全部验收要求；在工程层面，采用 FastAPI + SQLAlchemy + Vue3 的前后端分离架构，AI 模块以可插拔、可解释的双模式引擎落地，AI 建议与正式数据分离并形成人工确认闭环，知识图谱以统一端点 + Cytoscape.js 可视化呈现。

8.2 不足与展望

- 权限控制较简化**：当前以 `X-User-Id` 头识别用户、角色约束停留在数据层，后续可引入 JWT 与基于角色的接口级鉴权；
- AI 以轻量方法为主**：TF-IDF / 句向量在语义深度上有限，可接入更强的 embedding 或 LLM 关系抽取（云端模式已预留接口）；
- 检索能力**：可引入 SQLite FTS5 对论文标题/摘要/笔记做全文检索；
- 协作实时性**：当前为本地单机演示，未来可扩展为多人在线协同与跨设备同步；
- 图谱规模**：节点较多时可加入分层布局、聚类折叠与按需加载以提升可视化性能。

附：项目源码目录见仓库 **backend/** (FastAPI 后端) 与 **frontend/** (Vue3 前端) , 数据库文件 **backend/research_graph.db**